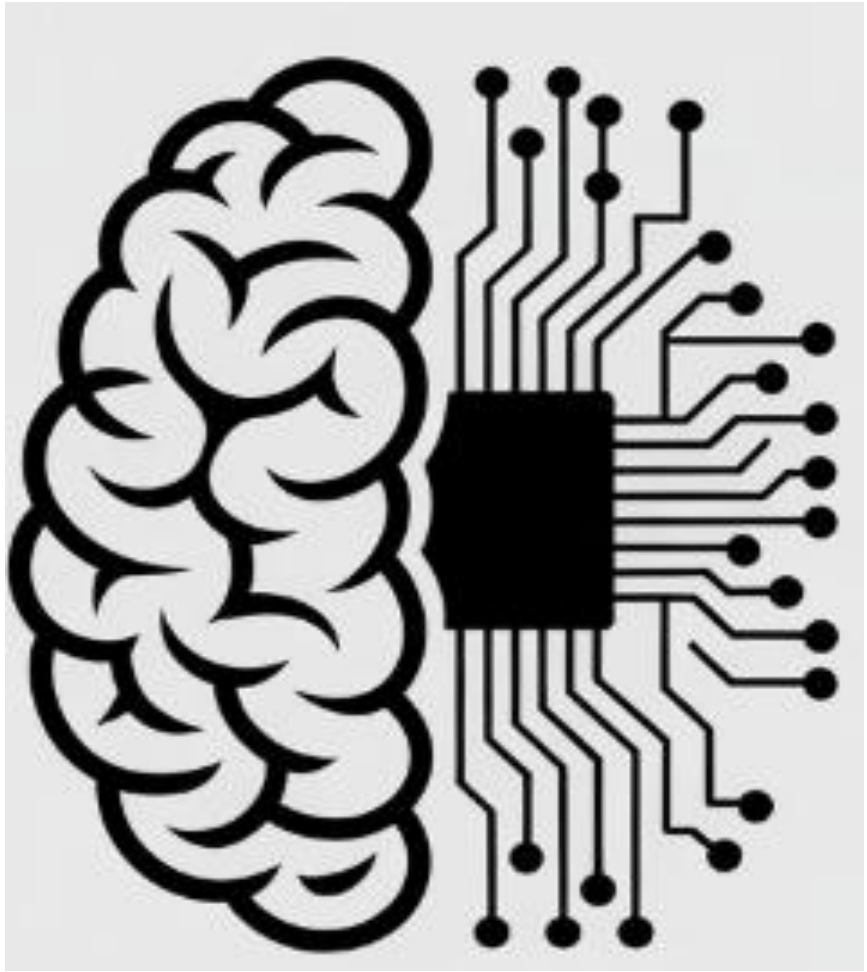


Informed Search/ Heuristic search

Dr. Sobia Tariq Javed

Heuristic Search

- Blind (uninformed) search techniques are termed “blind” because they do not exploit any knowledge about the problem structure, goal location, or state space. Instead, they systematically explore the search space until a solution is found.
- Heuristic search exploits problem, specific knowledge to improve search efficiency by guiding exploration toward promising states. It uses heuristics, rules of thumb that generally enhance decision-making by indicating which parts of the search space to examine first.

Heuristic Decision-Making Using Real-World Information

- **Shop with Several Checkouts**
- It is generally best to choose the checkout with the shortest queue.
- However, additional information can influence this decision.
  - A shorter queue may be slower if a customer has many items.
  - A slightly longer queue can be faster if customers have fewer items (e.g., fast checkout).
  - A checkout without a cashier should be avoided, even if the queue is shortest, as it will not lead to service.

Heuristic Decision-Making Using Real-World Information

Heuristics are rules of thumb that generally aid problem solving but do not guarantee the optimal solution; they are therefore fallible.

A heuristic represents an informed estimate of the next step, typically derived from experience or intuition.

When Are Heuristic Search Methods Applied?

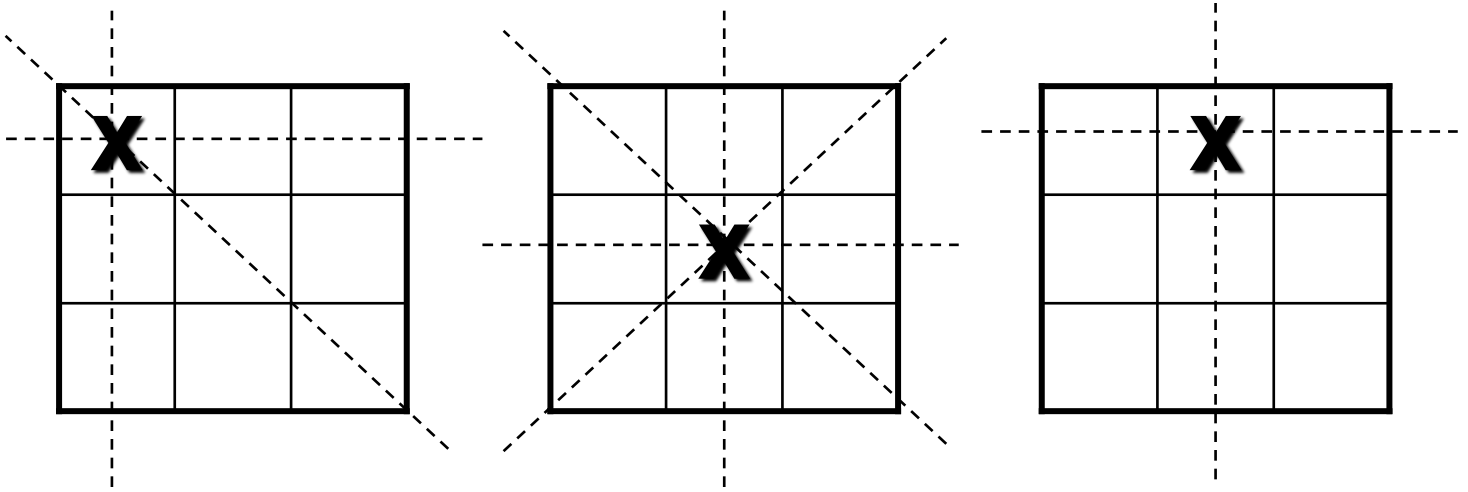
- Some problems do not admit a single exact solution due to inherent ambiguities in the problem formulation or in the available data.
- **Medical diagnosis** is a classic example: a given set of symptoms may correspond to multiple underlying conditions, so physicians select the most probable diagnosis and an appropriate treatment plan.

When Are Heuristic Search Methods Applied?

- **A problem may have an exact solution but computing it can be prohibitively expensive.**
- In many domains (e.g., chess), the state space grows explosively, increasing exponentially or factorially with search depth.
- Exhaustive search methods such as ***breadth-first*** and ***depth-first search*** often become impractical.
- Heuristic methods reduce this complexity by guiding the search toward the most promising paths.
- By pruning unpromising states and their descendants, heuristics help control state-space explosion and produce an acceptable solution efficiently.

How Are Heuristic Search Methods Applied?

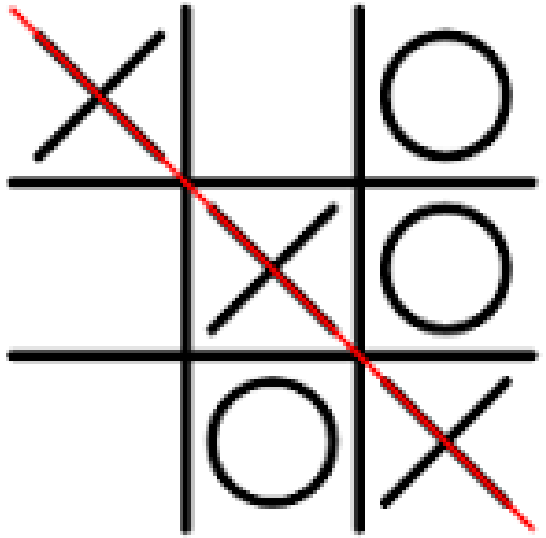
- A simple heuristic in the **game of tic-tac-toe** could be:
- We may move to the board in which X has the most winning lines



Three wins through a corner square

Four win through the center square

Two wins through a side square



Heuristic

- They are called Needle in Haystack problem

Finding the Needle

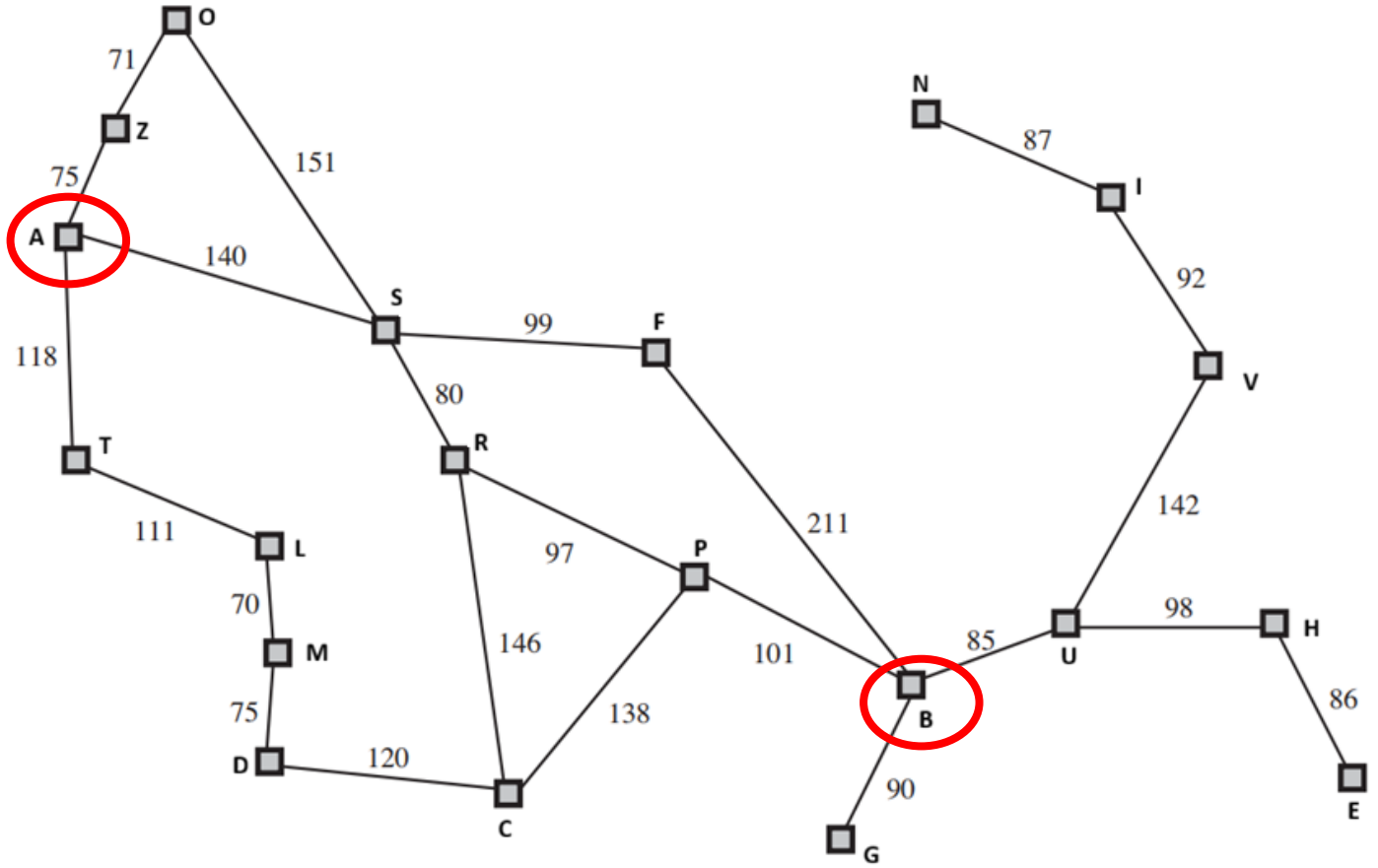
- Is Searching necessary?
- Not if we have magnet
- Question: is there some kind of mathematical analog to the magnet that pulls out the answer without having to search for?

Heuristic Function

- Previously, we discussed the **path cost** g(n), which represents the cost of reaching state n from the initial state.
- A **heuristic function** assigns a numerical value to each state, representing the “goodness” or promise of that node.
- Formally, h(n) denotes the estimated cost of the cheapest path from the current state n to the goal.

Map with path cost

- Suppose we want to find the shortest path from City A to City B.



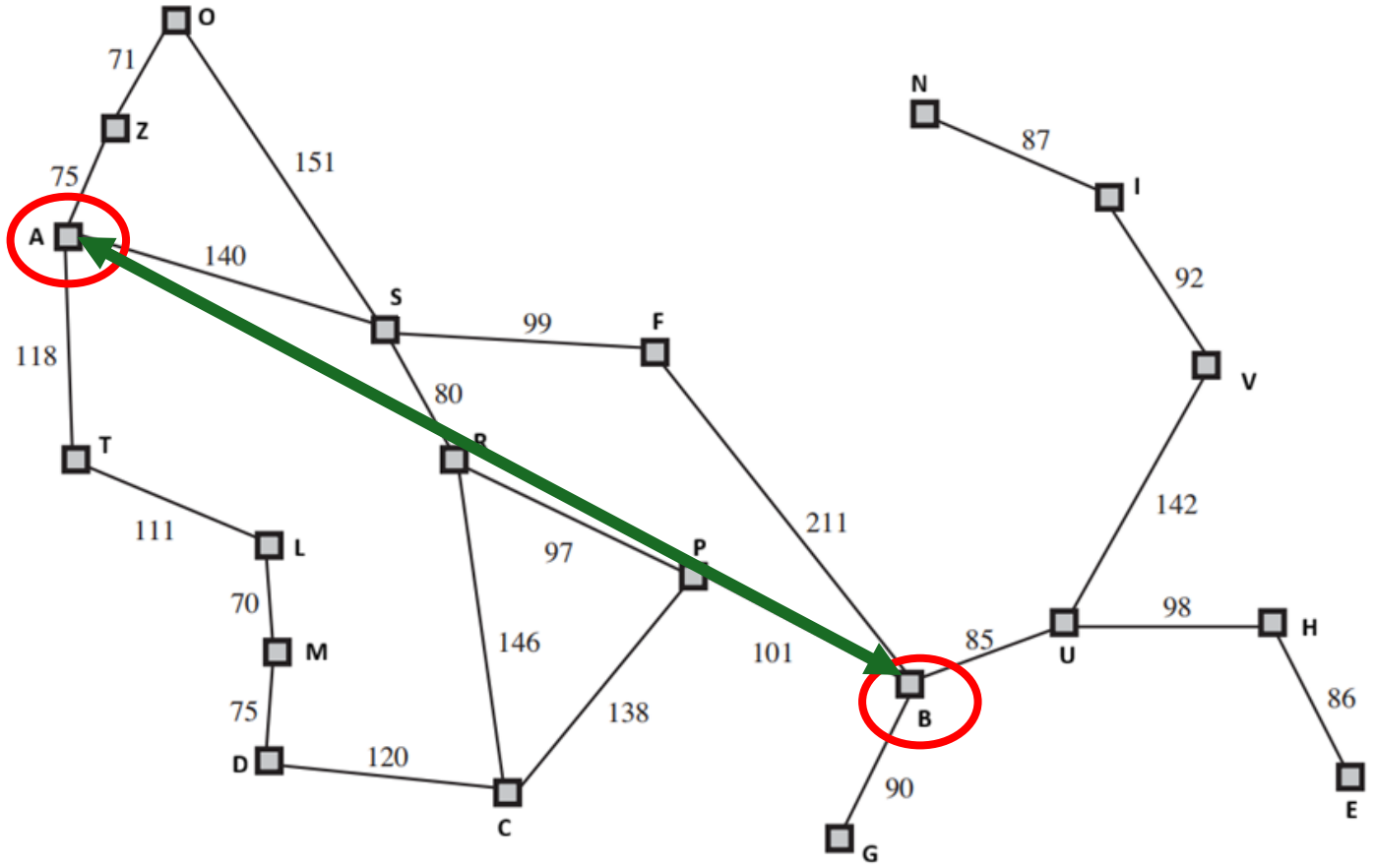
Heuristic Cost

- h(n) (**heuristic**) = *estimate of cost from n to goal*
- e.g., $h_{SLD}(n)$ = **straight-line distance** from n to B

City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

$h_{SLD}(n)$

- $h_{SLD}(n)$ = **straight-line distance** from n to B



Greedy best-first search

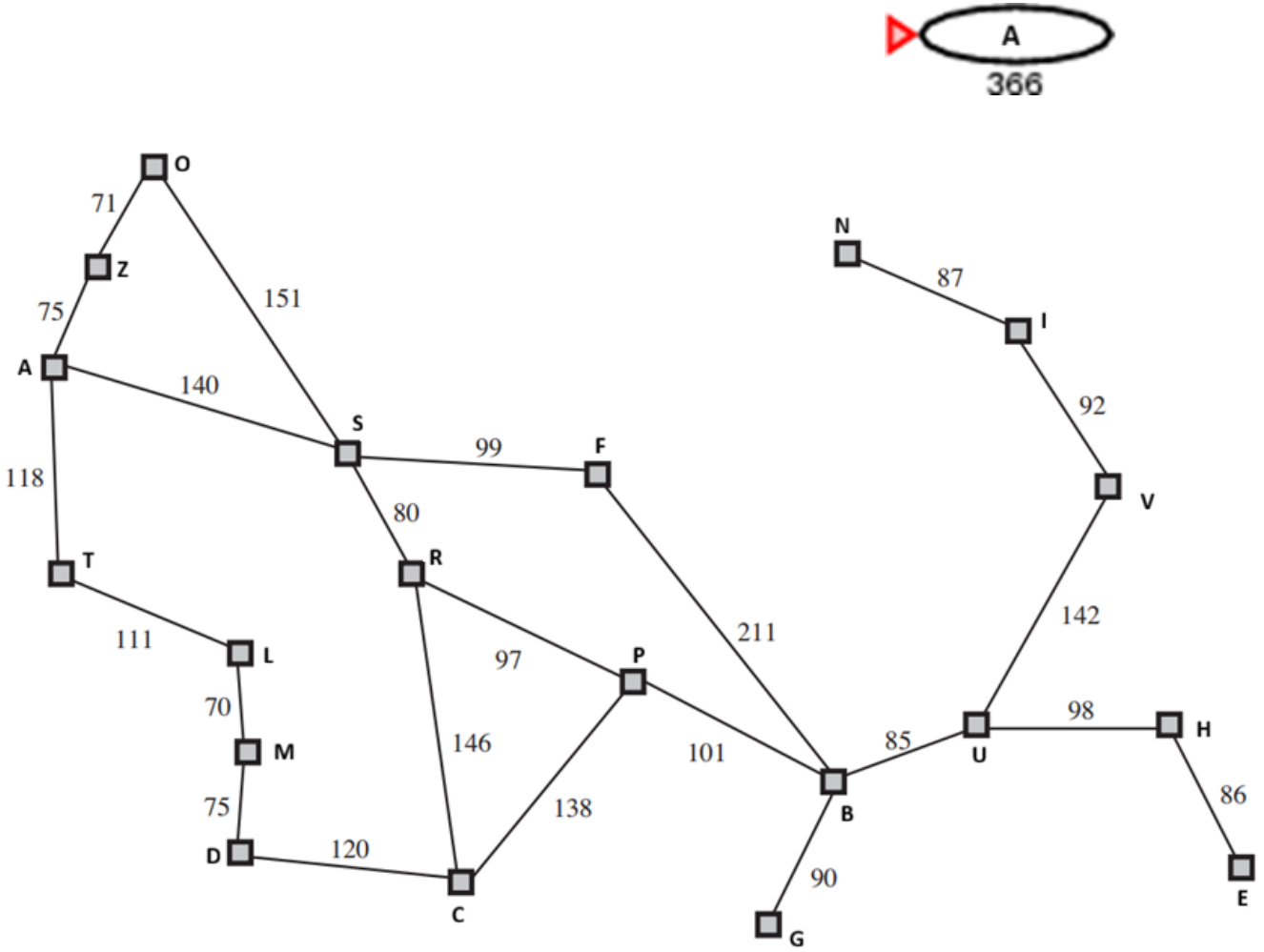
- Use evaluation function f(n) = h(n) instead of f(n) = g(n)
- Greedy best-first search expands the node that appears to be closest to goal f(n)=h(n).
- Fringe/ Frontier **F** is a **priority queue** sorted in ascending order of f(n) values.

Greedy best-first search - Algorithm

- **Input:** Initial node, goal test, heuristic function h(n)
- **Output:** Success (goal found) or Failure (no path)

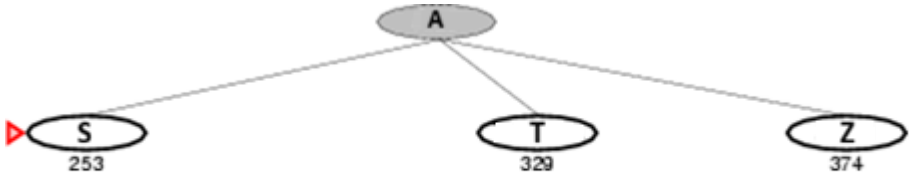
1. Initialize **F** as a priority queue containing the initial node, sorted by **f(n) = h(n)**
2. while **F** is not empty do:
   a. Remove the first node n from **F**
   b. if n is a goal node then
      return ***SUCCESS***
   c. else
      Generate all children of n
      Add children to **F**
      Sort **F** in ascending order of f-values (estimated cost to goal)
3. return ***FAILURE*** // F became empty without finding goal

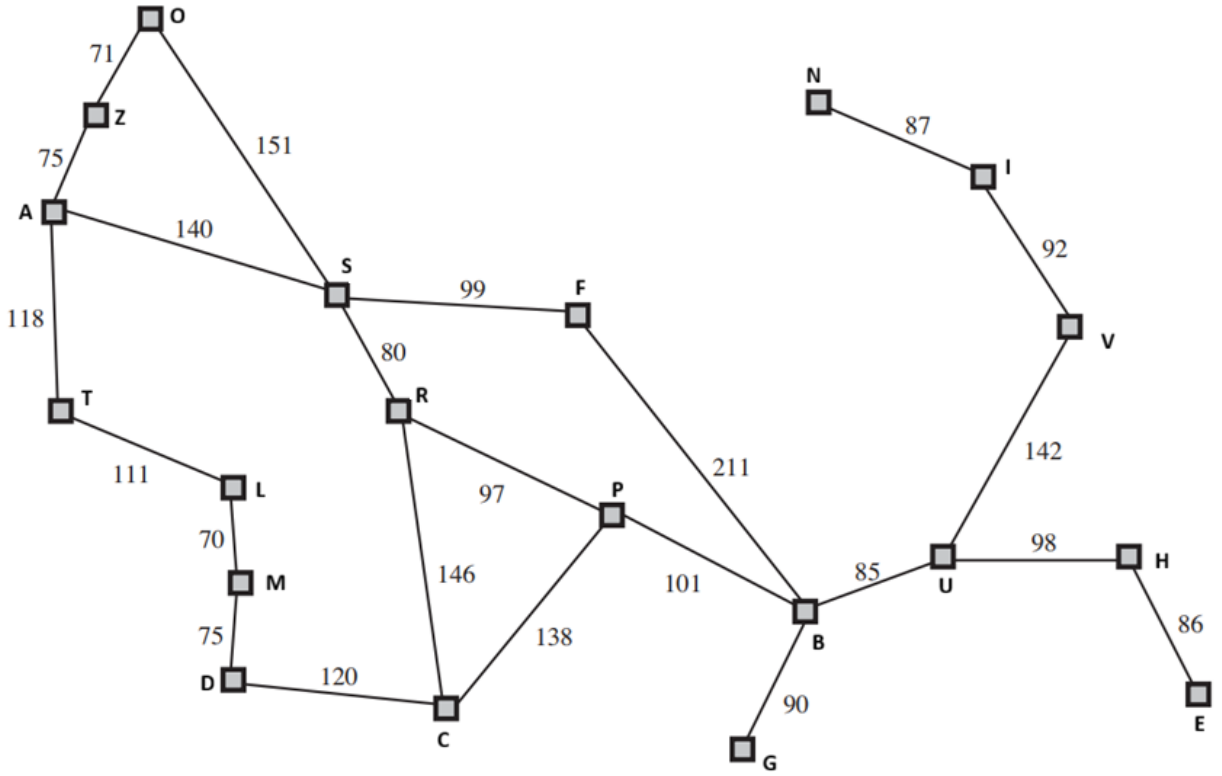
Greedy best-first search



City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

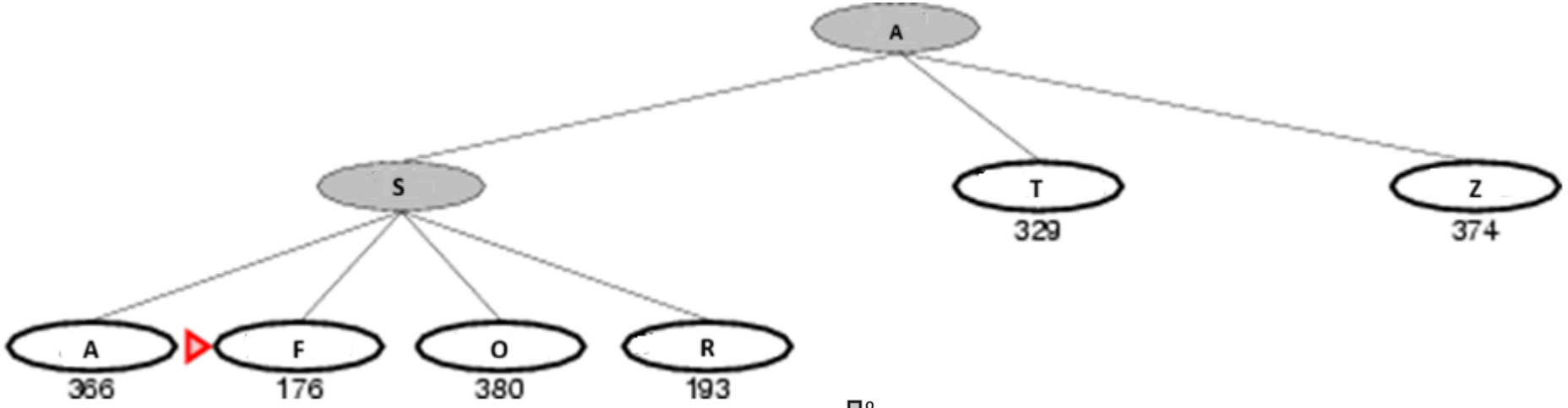
Greedy best-first search

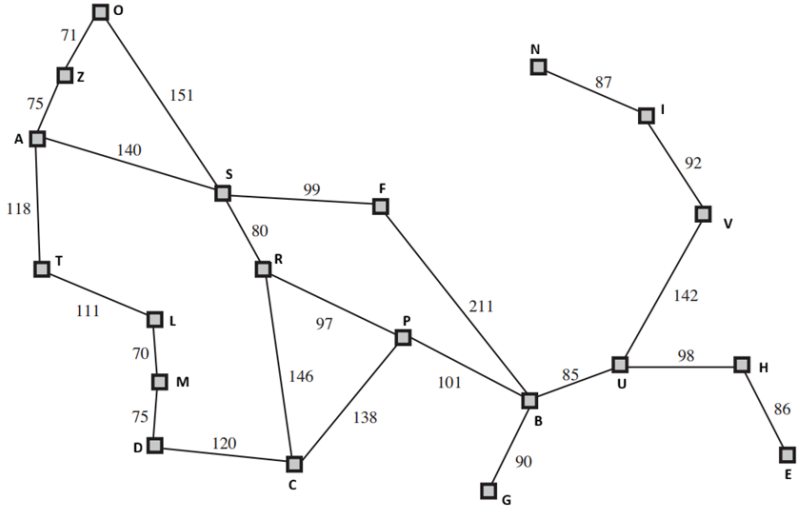




City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

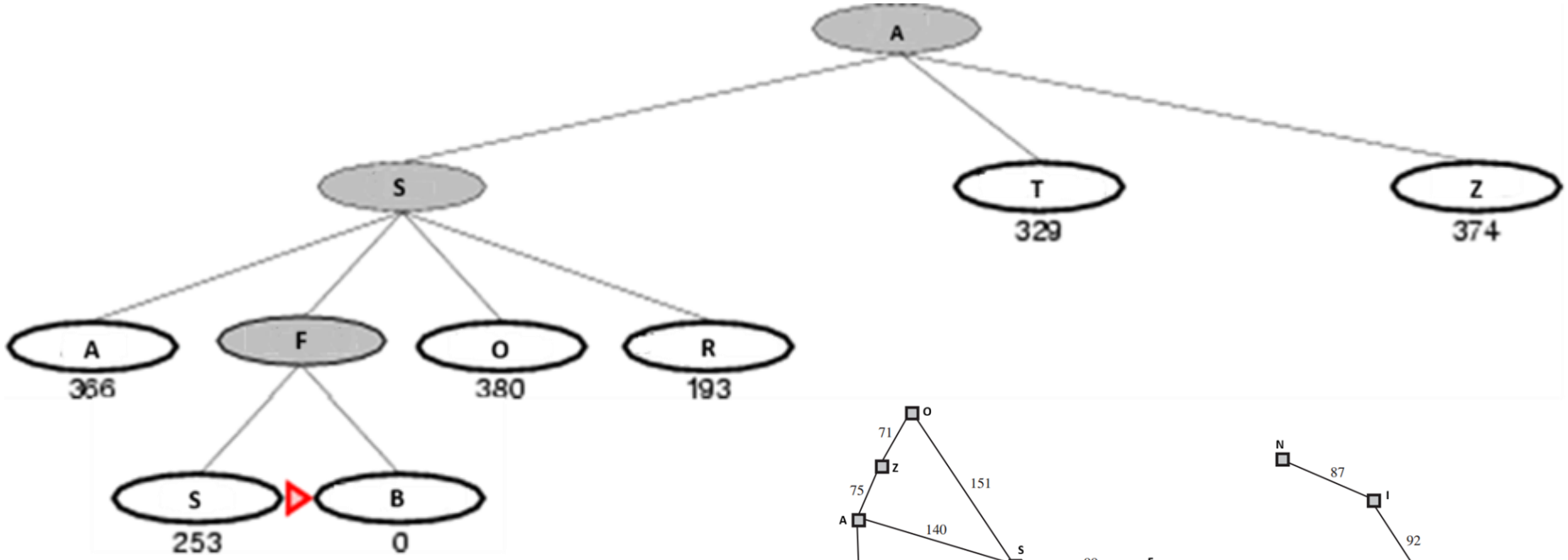
Greedy best-first search

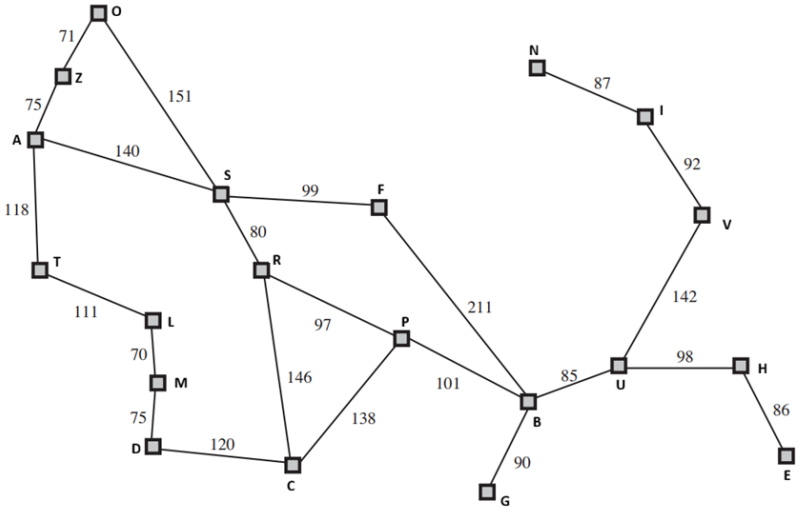




City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

Greedy best-first search

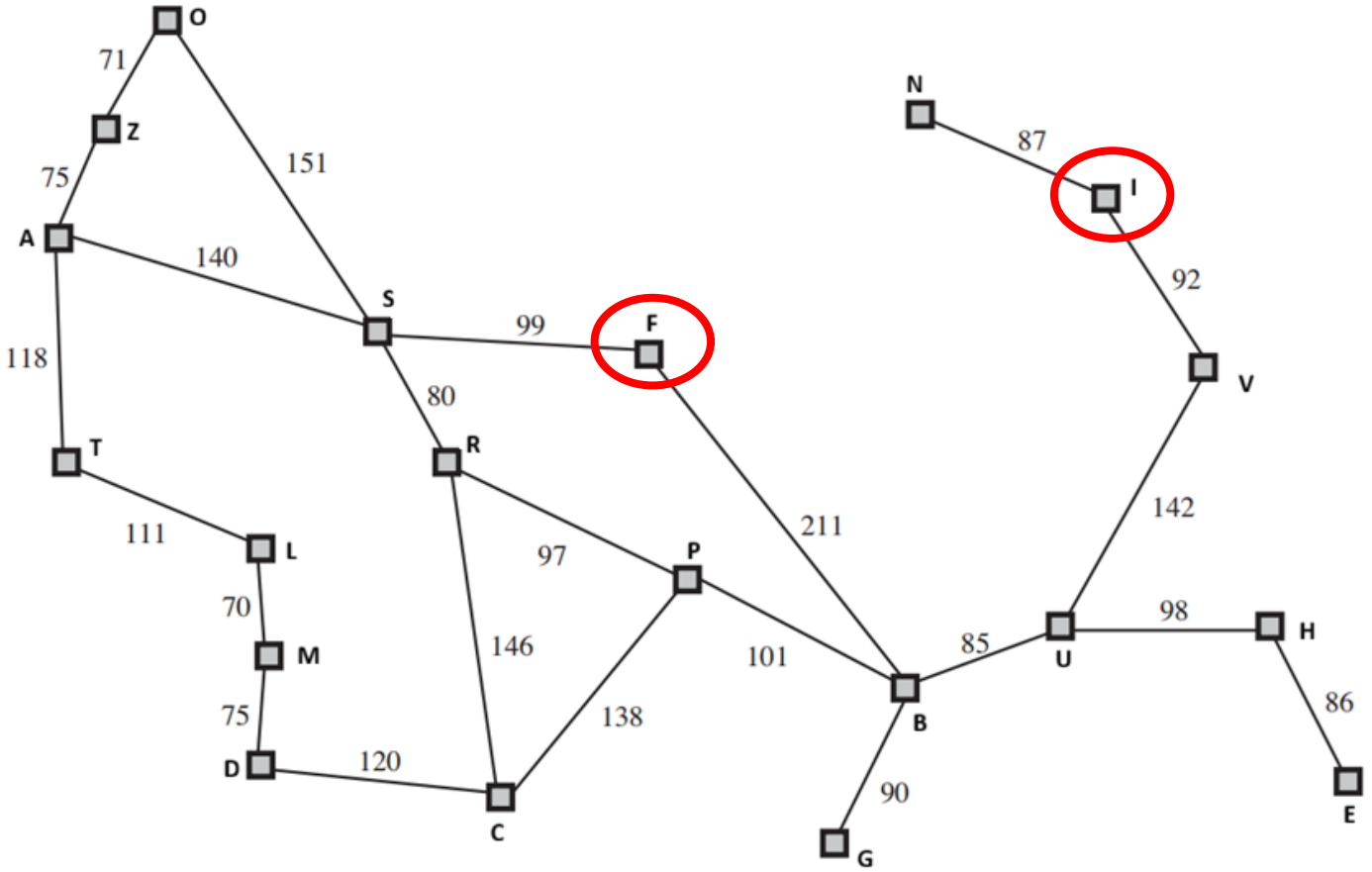




City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

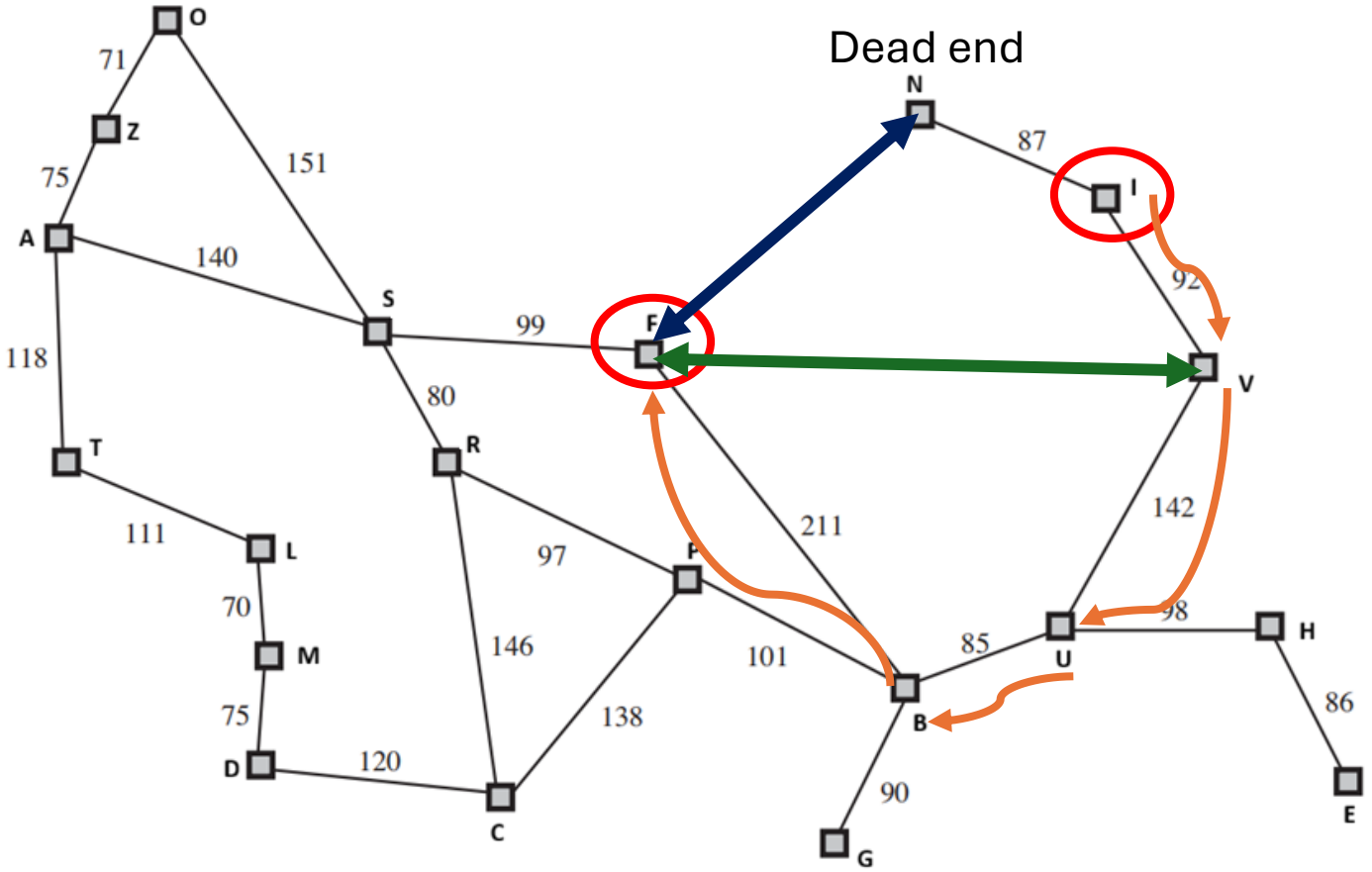
Greedy best-first search

- $h_{SLD}(n)$ = **straight-line distance** from n to B
- Source = I
- Destination = F



Greedy best-first search

- $h_{SLD}(n)$ = **straight-line distance** from n to B
- Source = I
- Destination = F



Greedy best-first search

- **Complete**? No – can get stuck in loops, e.g., I→N→I→N
- **Time**? $O(b^m)$, but a good heuristic can give dramatic improvement
- **Space**? $O(b^m)$ -- keeps all nodes in memory
- **Optimal**? No

Greedy best-first search

- **Vacuum Cleaner Example on board**

A* search

- Idea: avoid expanding paths that are already expensive
- Evaluation function f(n) = g(n) + h(n)
- g(n) = cost from start node to n
- h(n) = estimated cost from n to goal node
- f(n) = estimated total cost of path through n to goal

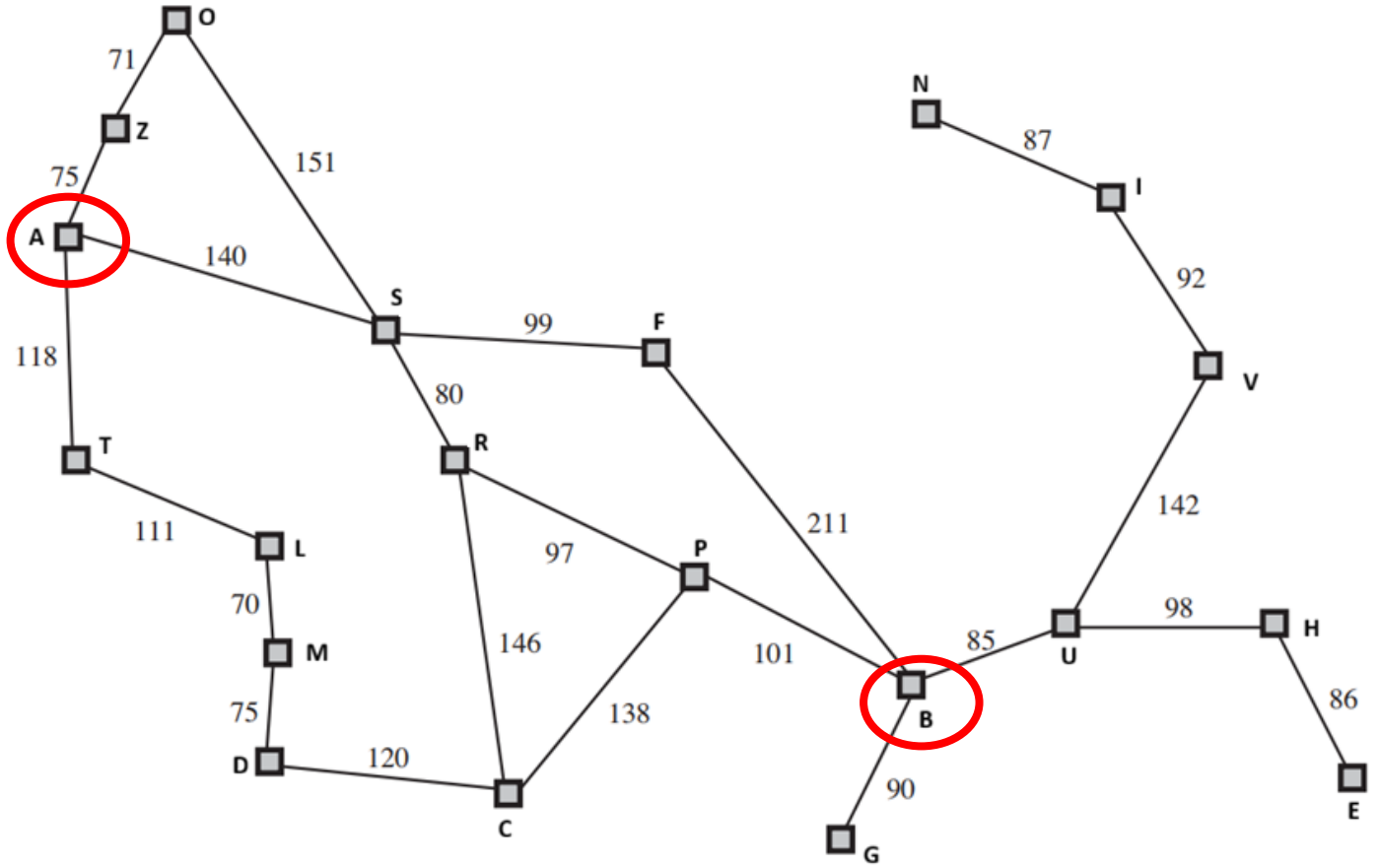
A* search

- **Input:** Initial node, goal test, heuristic function h(n)
- **Output:** Success (goal found) or Failure (no path)

1. Initialize **F** as a priority queue containing the initial node, sorted by **f(n) = g(n) + h(n)**
2. while **F** is not empty do:
   a. Remove the first node n from **F**
   b. if n is a goal node then
      return ***SUCCESS***
   c. else
      Generate all children of n
      Add children to **F**
      Sort **F** in ascending order of f-values (estimated total cost to goal)
3. return ***FAILURE*** // F became empty without finding goal

Map with path cost

- Suppose we want to find the shortest path from City A to City B.

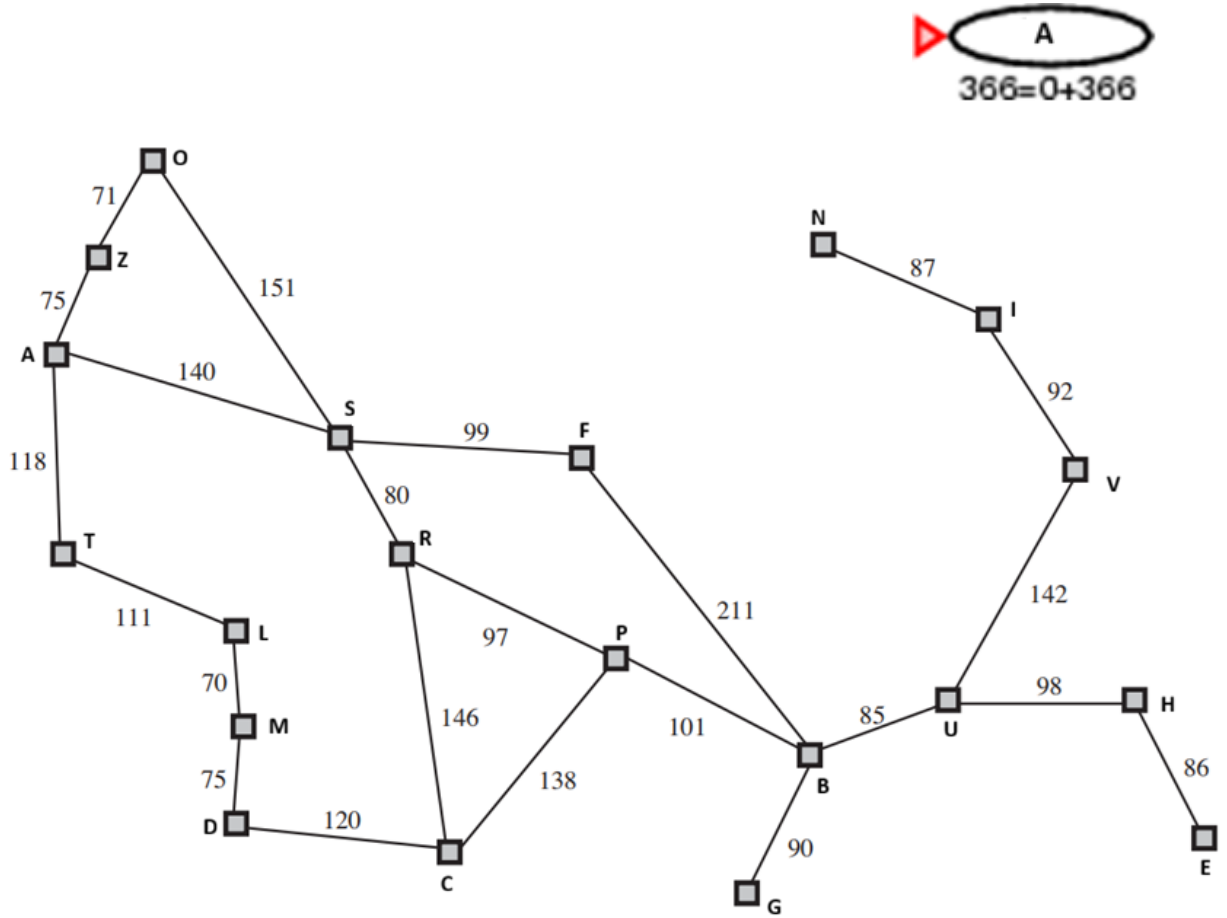


Heuristic Cost

- h(n) (**heuristic**) = *estimate of cost from n to goal*
- e.g., $h_{SLD}(n)$ = **straight-line distance** from n to B

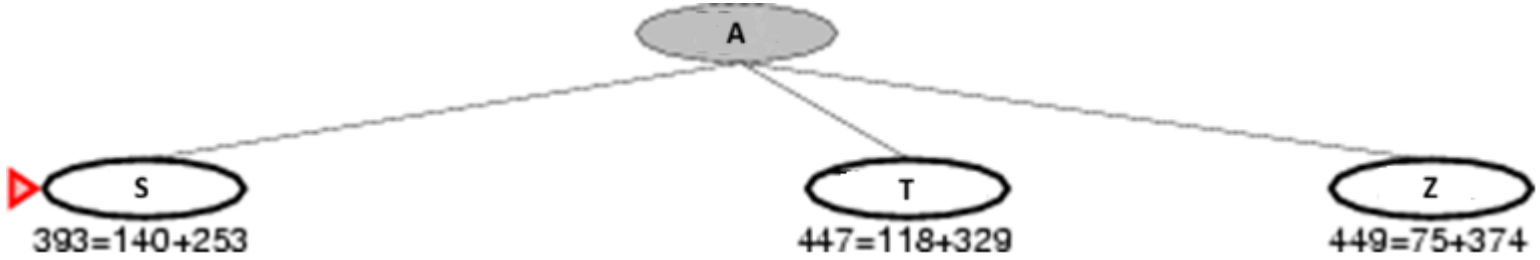
City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

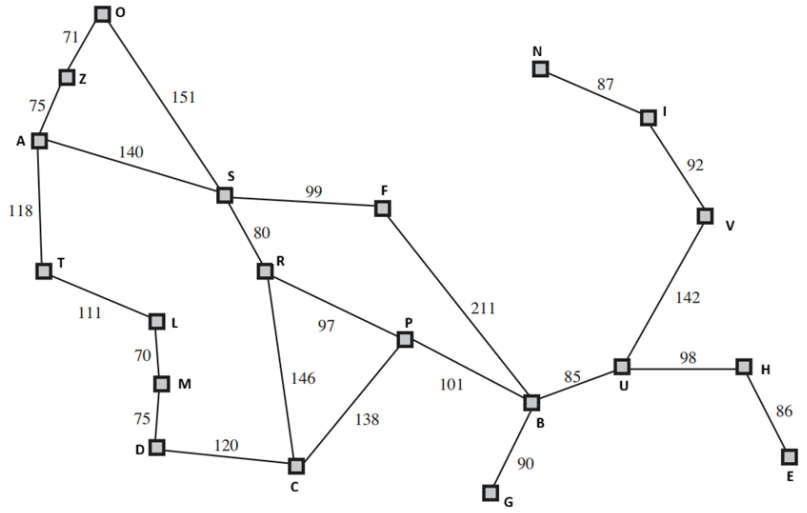
A* search



City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

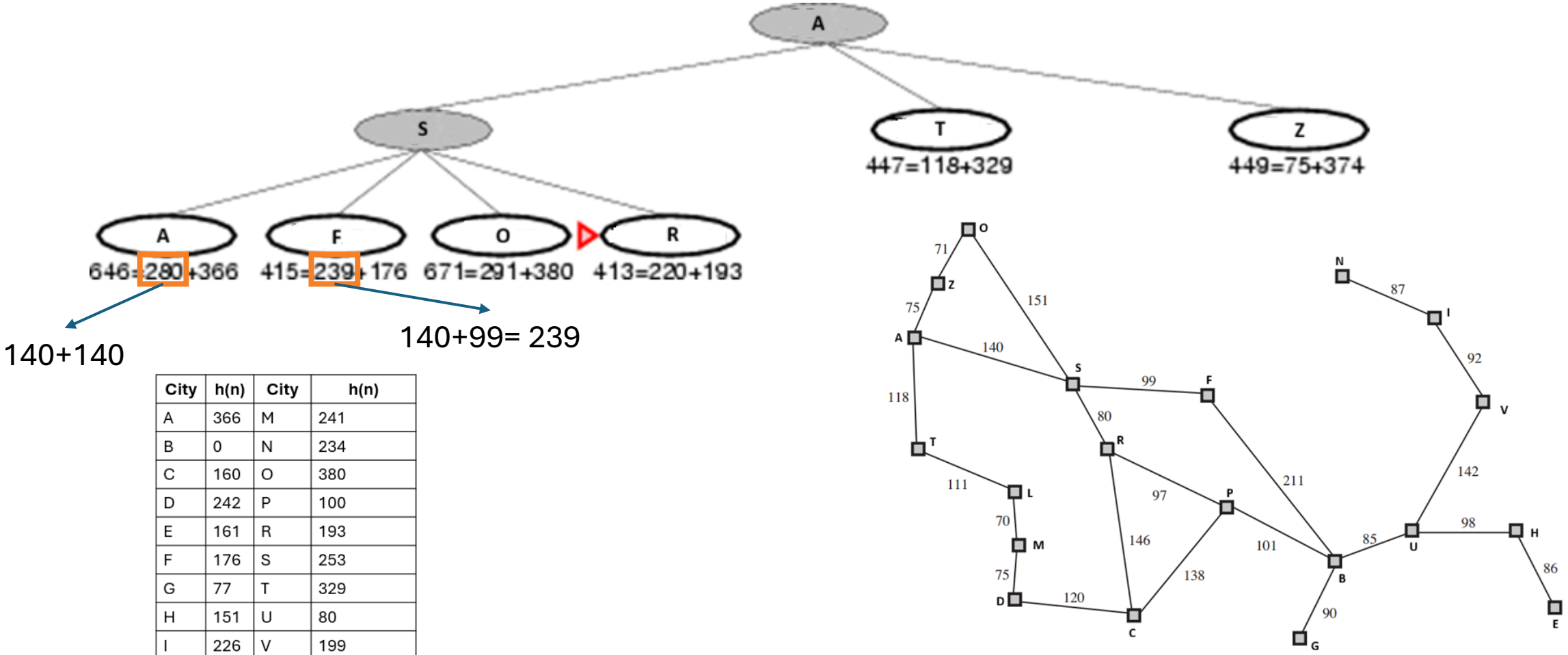
A* search





City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

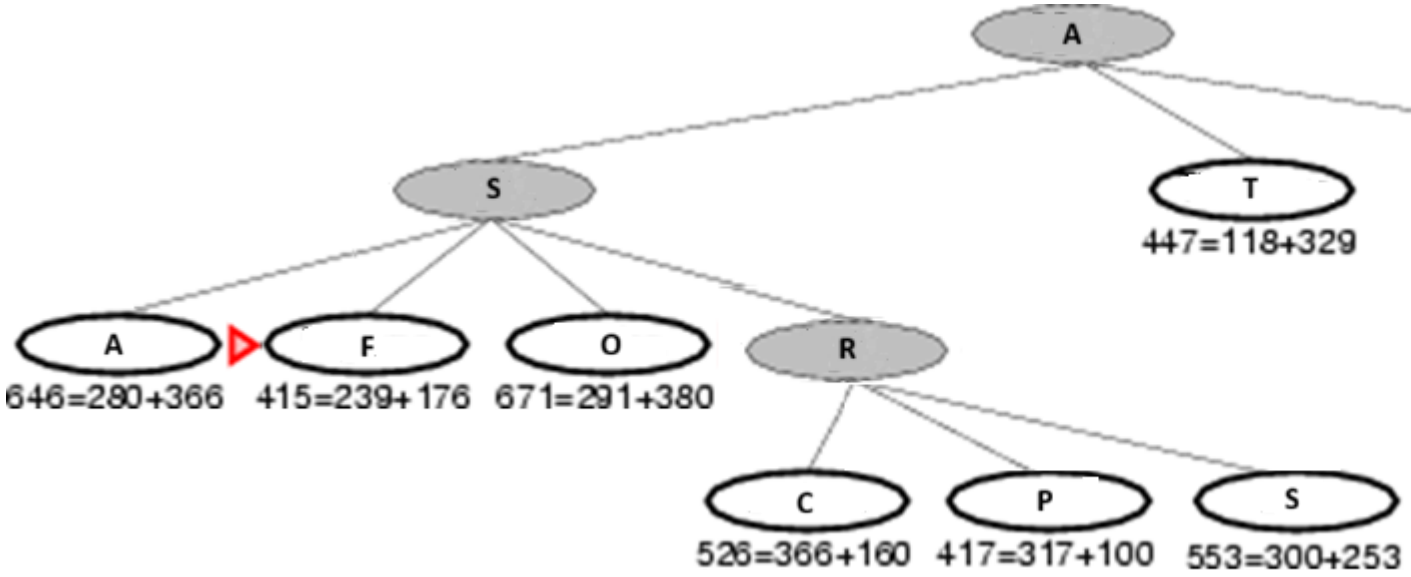
A* search

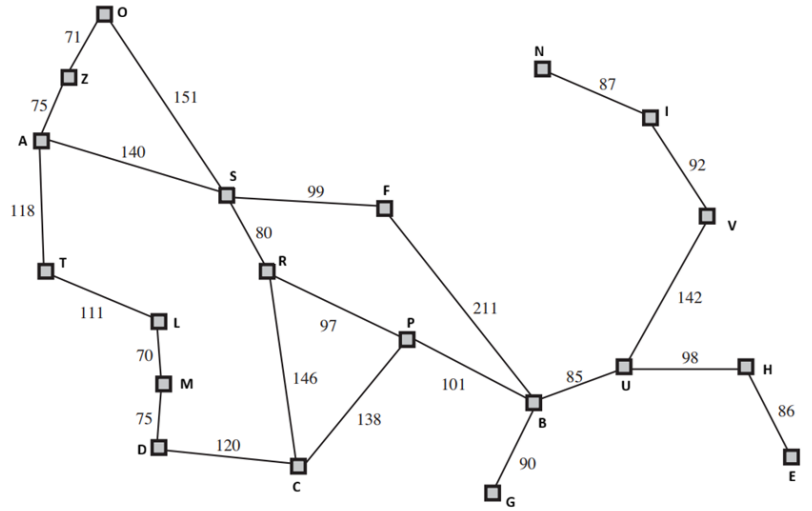


City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

A* search

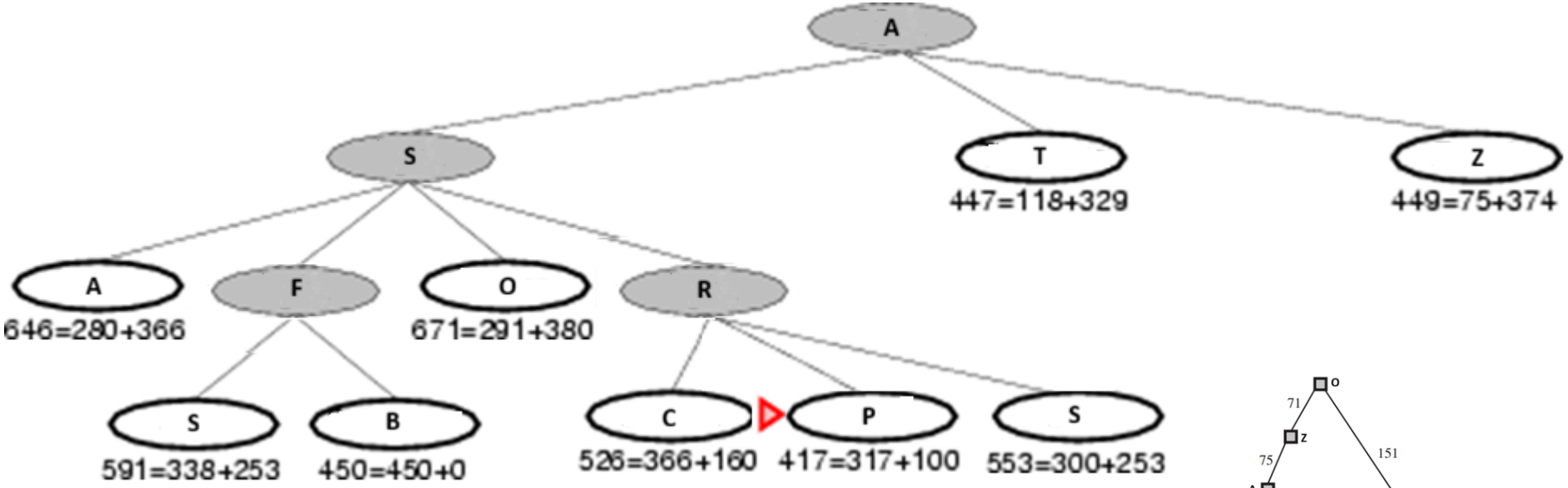
City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

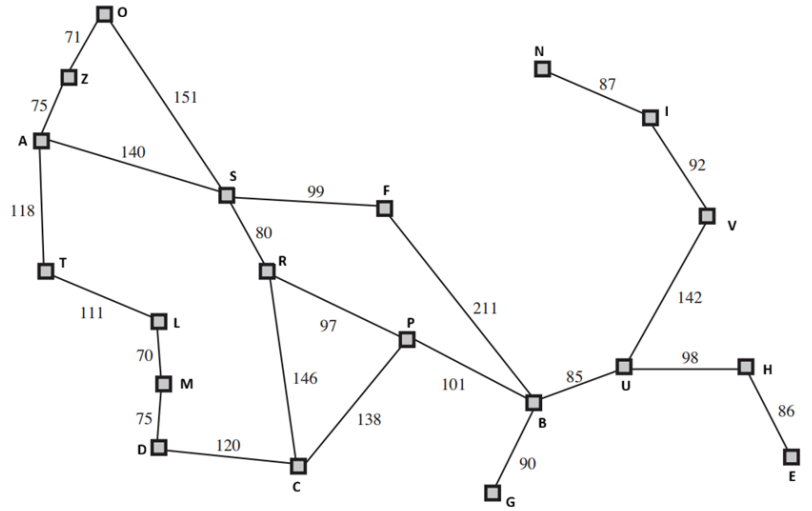




A* search

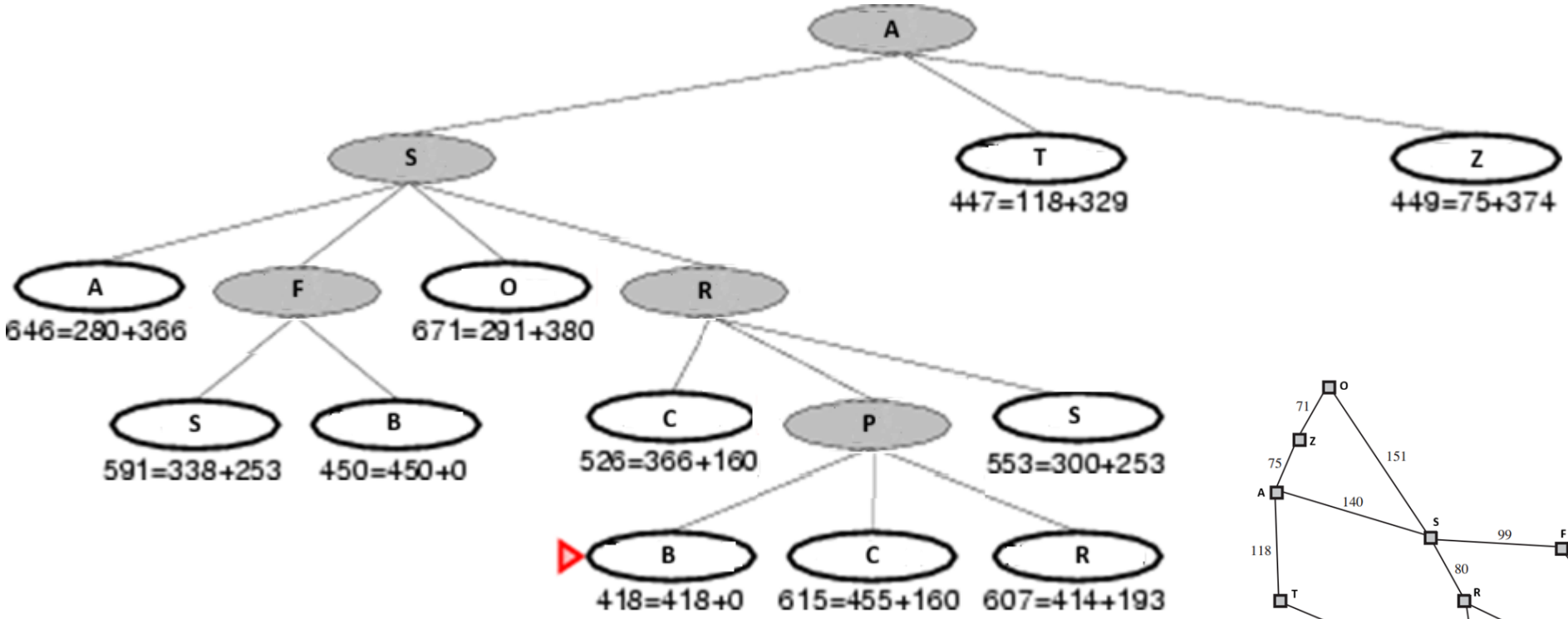
City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374

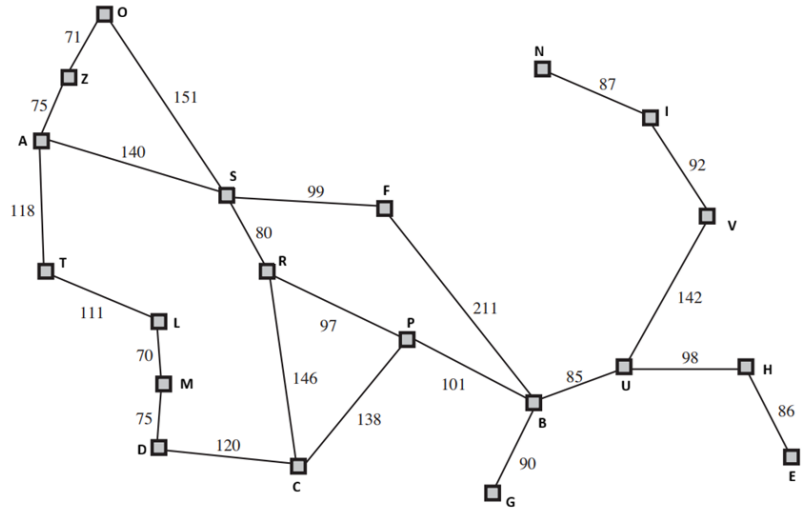




A* search

City	h(n)	City	h(n)
A	366	M	241
B	0	N	234
C	160	O	380
D	242	P	100
E	161	R	193
F	176	S	253
G	77	T	329
H	151	U	80
I	226	V	199
L	244	Z	374



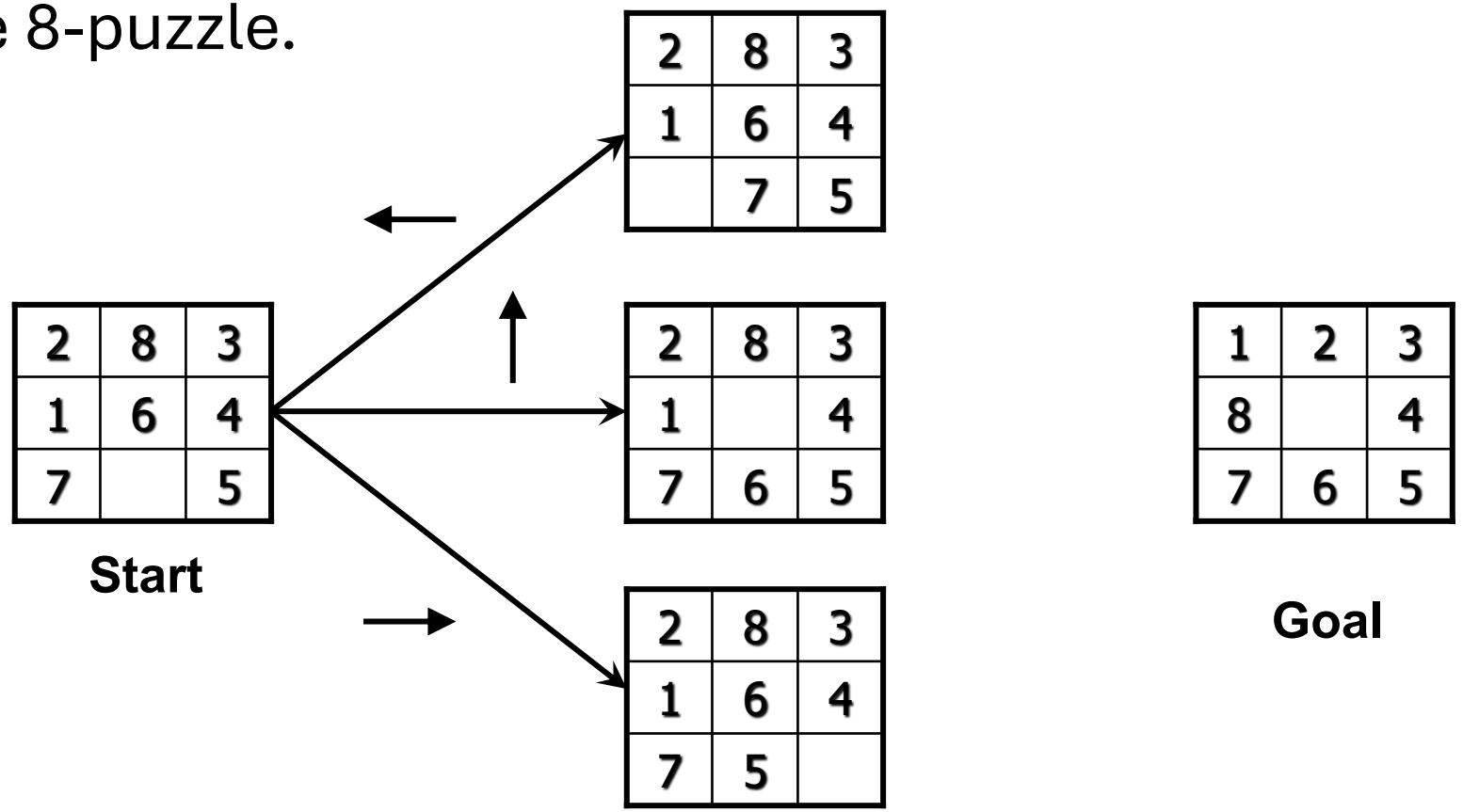


A* search

- **Vacuum Cleaner Example on board**

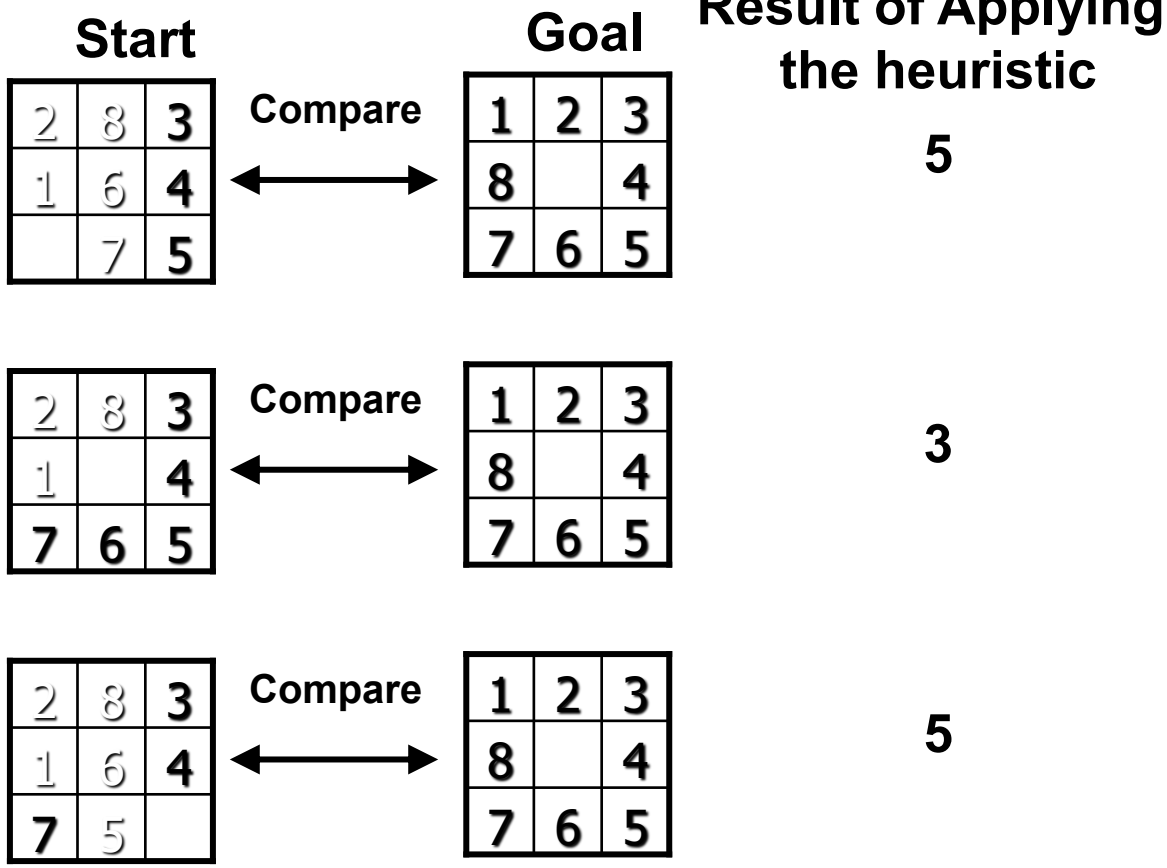
8-Puzzle with Heuristic Search

- Let's look at the performance of several different heuristics for solving the 8-puzzle.



8-Puzzle with Heuristic Search

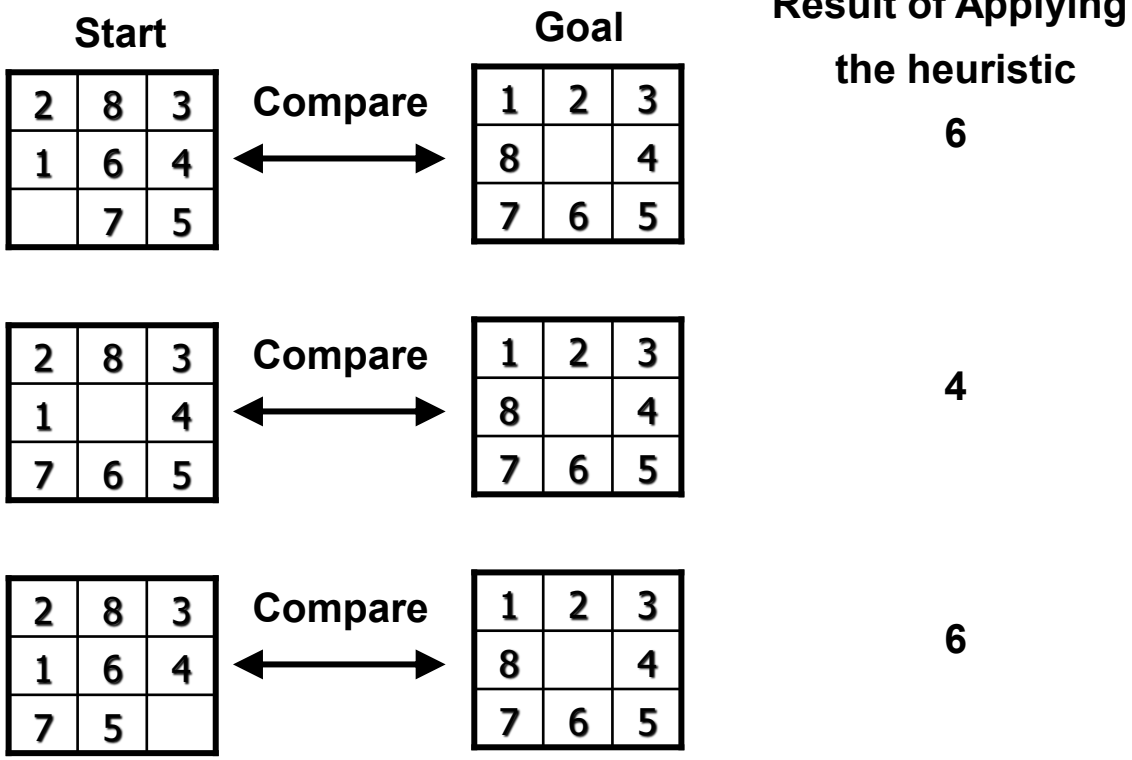
- **Heuristic #1:** Count the tiles out of place in each state when it is compared with the goal.



8-Puzzle with Heuristic Search

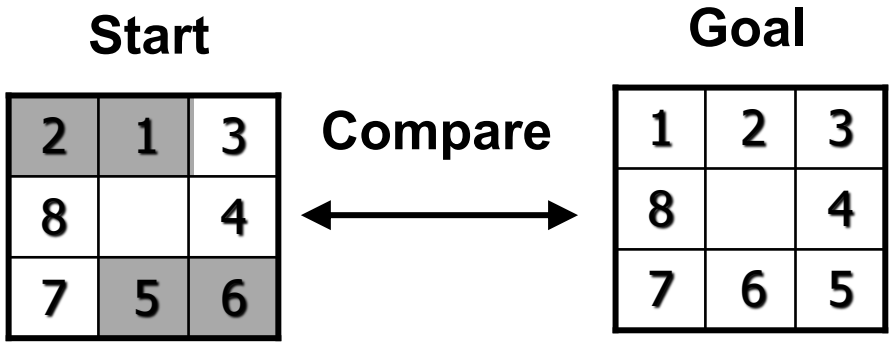
- **Heuristic #2:** Sum all the distances by which the tiles are out of place (one for each square, a tile be moved to reach its position in the goal state).

Cost of moving 2 to location (1,2) is 1
Cost of moving 1 to location (1,1) is 1
Cost of moving 8 to location (2,1) is 2
Cost of moving 6 to location (3,2) is 1
Cost of moving 7 to location (3,1) is 1
Total Cost = 1+1+2+1+1=6



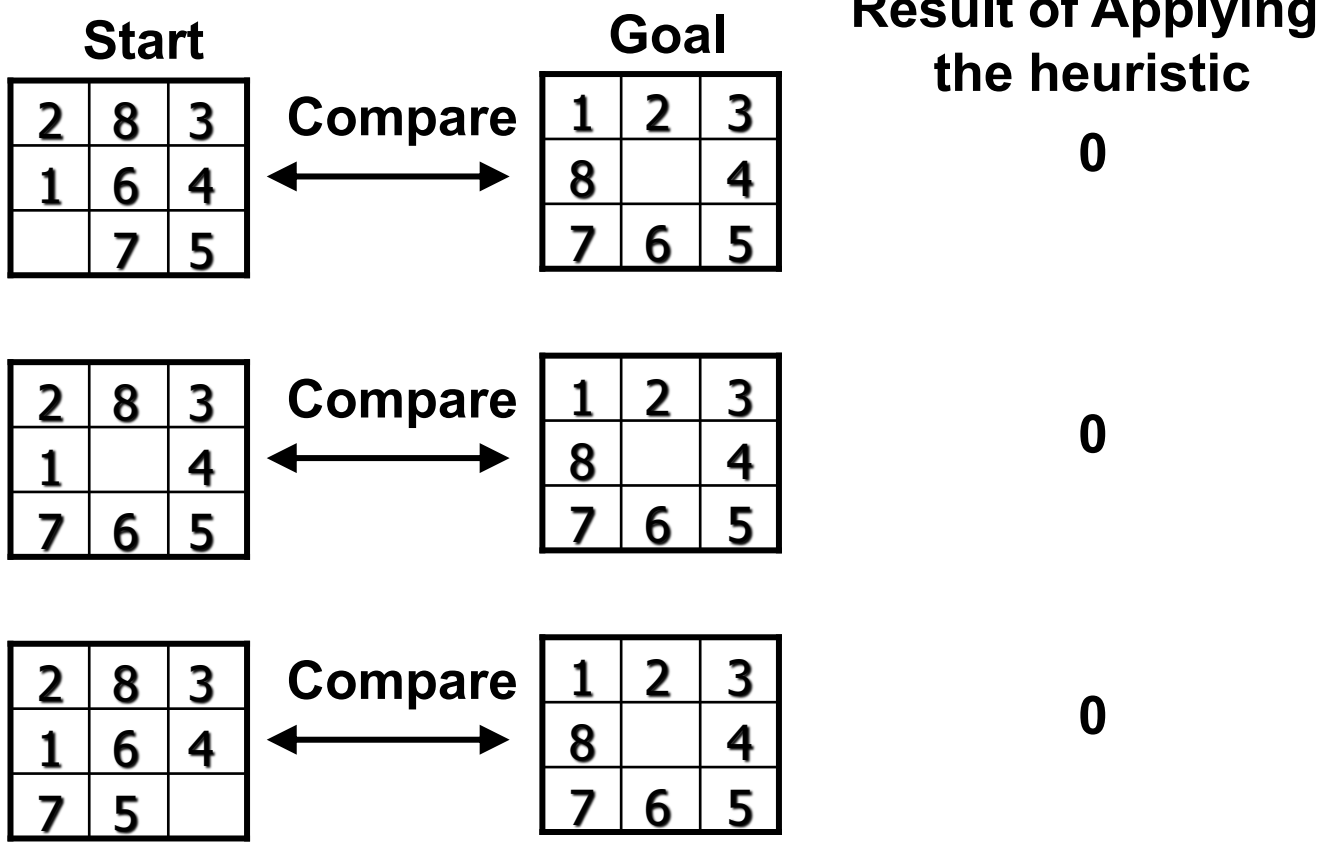
8-Puzzle with Heuristic Search

- **Heuristic #3:** Both above heuristics can be criticized for failing to acknowledge the difficulty of tile reversals.
- Tile reversals means if two tiles are next to each other and the goal requires their being in opposite locations.
- It takes (many) more than two moves to put them back in place, as the tile must "go around" each other
- The following figure shows an 8-puzzle with a goal and two tile reversals, i.e. 1 and 2; 5 and 6.



8-Puzzle with Heuristic Search

- **Heuristic #3 (continued):** A heuristic that takes into account could be to multiply a small number (say 2) with each direct tile reversal (i.e. where two adjacent tiles must be exchanged to be in the order of the goal.



8-Puzzle with Heuristic Search

- A **fourth heuristic**, which may overcome the limitation of the tile reversal heuristic could be: add the sum of distances out of place and 2 times the number of direct tile reversals.
- It must be noted that each of the above heuristic ignores some critical bit of information and is subject to improvements.

8-Puzzle with Heuristic Search

- Let's solve 8-puzzle now with h(n) being the number of tiles out of place. The evaluation function **f(n) = g(n) + h(n)** thus in this case would be composed of
- g(n) = actual distance from n to the start state.
- h(n) = number of tiles out of place.

A* algorithm- 8-Puzzle

- Let's solve 8-puzzle now with h(n) being the number of tiles out of place. The evaluation function **f(n) = g(n) + h(n)** thus in this case would be composed of
- g(n) = actual distance from n to the start state.
- h(n) = number of tiles out of place.

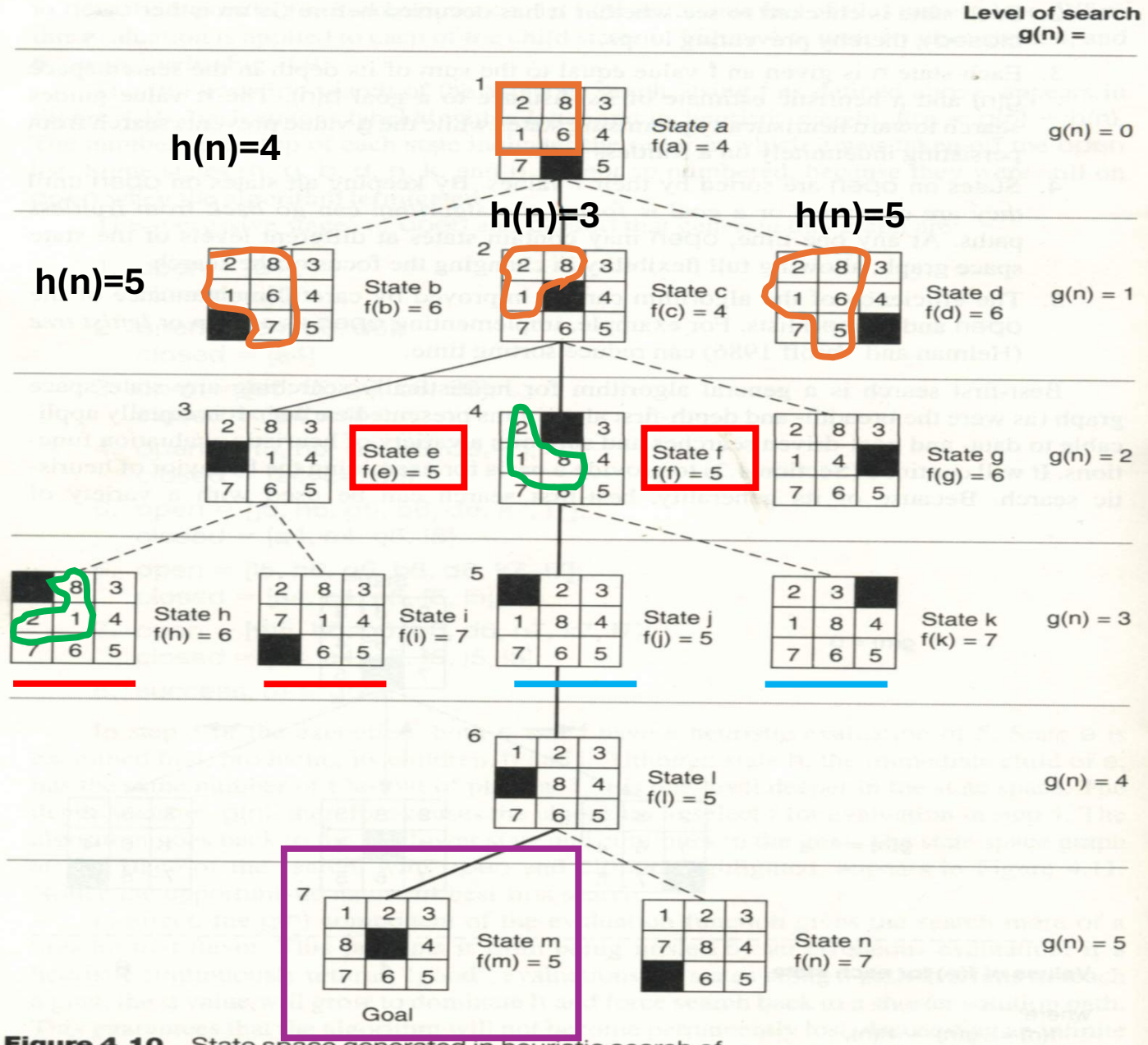


Figure 4.10 State space generated in heuristic search of the 8-puzzle graph.

Conditions for Optimality

- Admissible
- Consistency

A* is Admissible

- A* will never overestimate the cost of reaching the goal.
- The cost to reach goal is guaranteed to be lower or equal to the actual cost.
- This property ensures that the algorithm will eventually find the optimal solution, if one exists.

Admissible heuristics

- A heuristic h(n) is **admissible** if for every node n,

  h(n) ≤ h*(n), where h*(n) is the **true/actual** cost to reach the goal state from n and h(n) is the estimated cost to reach the goal

- An admissible heuristic **never overestimates** the cost to reach the goal, i.e., it is **optimistic**

- Example: $h_{SLD}(n)$ (never overestimates the actual road distance)

- Theorem: If h(n) is admissible, A* using TREE-SEARCH is optimal

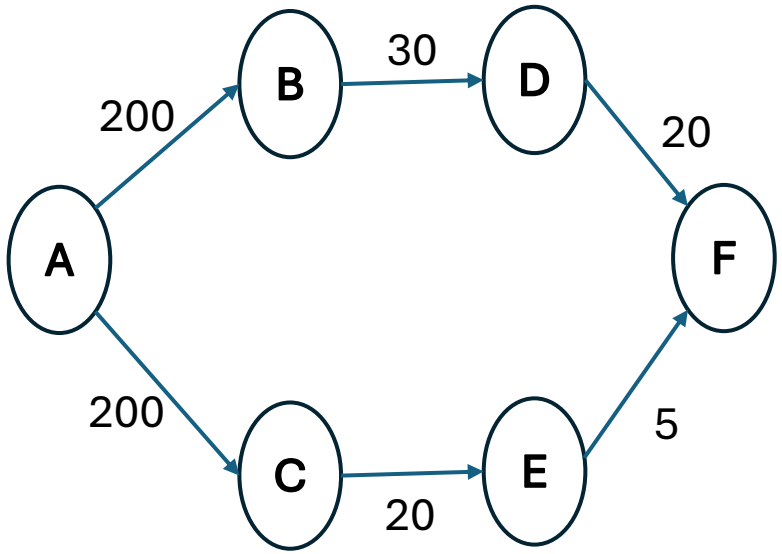
Admissible heuristics

- Find the shortest path
- Source = A
- Goal = F

- According UCS path cost

  A→ C → E → F = 225 **Best**

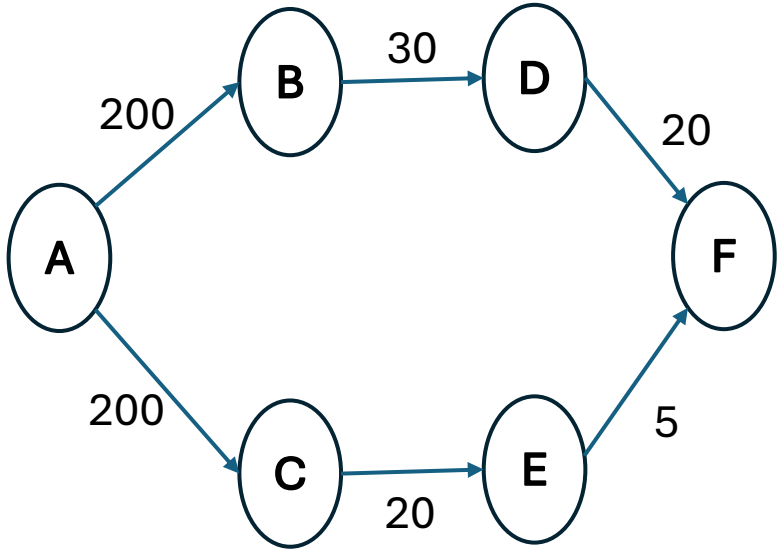
  A→ B → D → F = 250



Admissible heuristics

- Underestimated heuristic

City (n)	h(n)	City (n)	h(n)
A	35	D	10
B	20	E	5
C	25	F	0



- Apply A*

A→ C → E → F

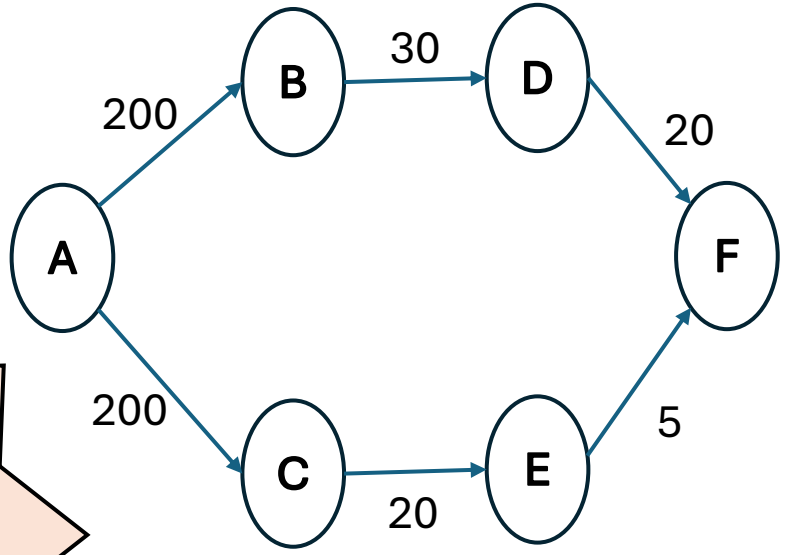
Admissible heuristics

- Overestimated heuristic

City (n)	h(n)	City (n)	h(n)
A	35	D	10
B	70	E	10
C	80	F	0

- Apply A*

A→ B → D → F

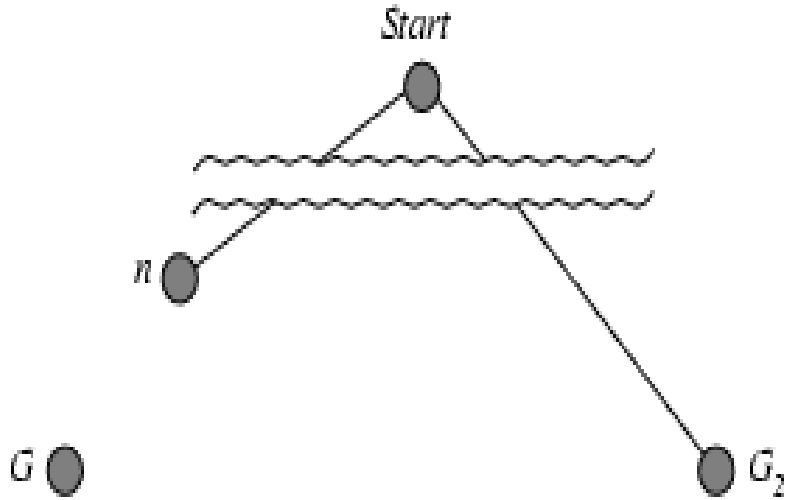


Never Overestimate the heuristic

Optimality of A* (proof)

A suboptimal goal is a solution that works, but is not the best possible one. You reach the goal, but not in the shortest, cheapest, or most efficient way.

- Suppose some suboptimal goal $\mathbf{G_2}$ has been generated and is in the fringe/Frontier. Let **n** be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal **G**.
- **f(n) = g(n) + h(n)**
- $f(G_2) = g(G_2)$ since $h(G_2) = 0$
- $g(G_2) > g(G)$ since G_2 is suboptimal
- $f(G) = g(G)$ since $h(G) = 0$
- $f(G_2) > f(G)$ from above

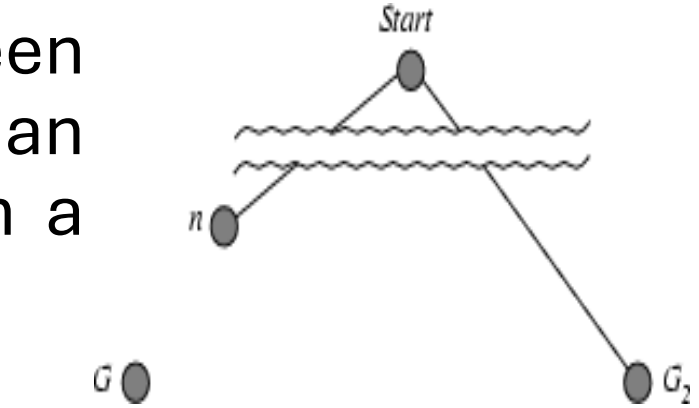


$f(n) = g(n) + h(n)$

A suboptimal goal is a solution that works, but is not the best possible one. You reach the goal, but not in the shortest, cheapest, or most efficient way.

Optimality of A* (proof)

- Suppose some suboptimal goal $\mathbf{G_2}$ has been generated and is in the fringe/Frontier. Let **n** be an unexpanded node in the fringe such that n is on a shortest path to an optimal goal **G**.



- $f(G_2) > f(G)$ from above (just proved)
- $h(n) \leq h^*(n)$ since h is admissible, h* is minimal distance/actual cost, h(n) is the estimated cost to reach the goal
- $g(n) + h(n) \leq g(n) + h^*(n)$
- $f(n) \leq f(G) < f(G_2)$

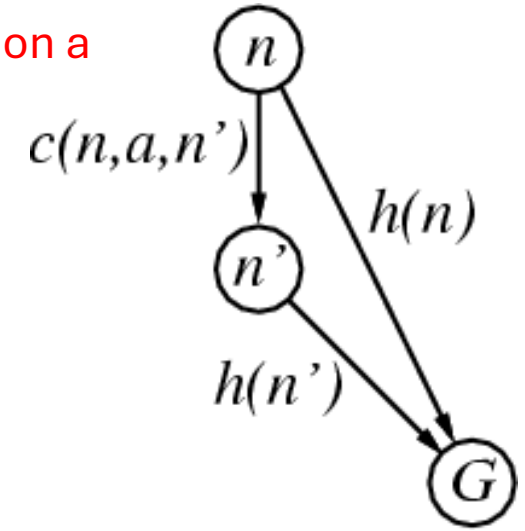
Hence $f(G_2) > f(n)$, and A* will never select G_2 for expansion

Consistent heuristics

- A heuristic is consistent if for every node *n*, every successor *n'* of *n* generated by any action *a*, the estimated cost of reaching the goal from node n, *h(n),* is not greater than the step cost of getting to *n'*, (g(n'))+ *estimated cost of reaching the goal from n' (f(n'))*

c(n,a,n') = Actual cost of reaching n' via n with action a

- $h(n) \leq g(n') + h(n')$
- $h(n) \leq c(n,a,n') + h(n')$ → *(triangle inequality)*



- Intuition: can't do worse than going through n'.

c(n,a,n') = Actual cost of reaching n' via n with action a

A heuristic is consistent if for every node *n*, every successor *n'* of *n* generated by any action *a*, the estimated cost of reaching the goal from node n, *h(n),* is not greater than *(f(n')) [*the step cost of getting to *n'*, (g(n'))+ *estimated cost of reaching the goal from n' h(n')]*

Consistent heuristics

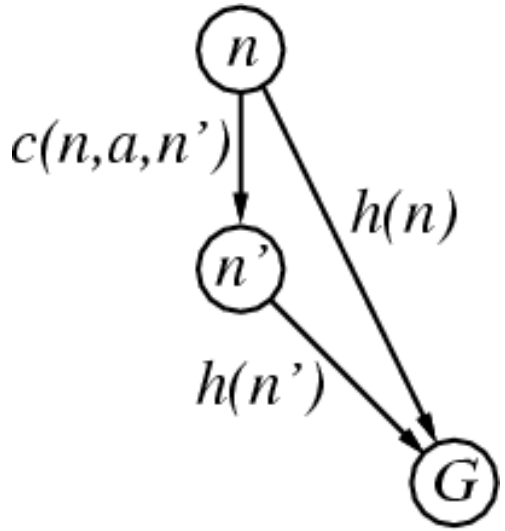
- $h(n) \leq g(n') + h(n')$
- $h(n) \leq c(n,a,n') + h(n')$

g(n') = g(n) + c(n, a, n')

- Intuition: <u>can't do worse than going through n'.</u>
- If *h* is consistent, we have

$$f(n') = g(n') + h(n') = g(n) + c(n,a,n') + h(n')$$
$$= g(n) + [c(n,a,n') + h(n')]$$
$$\geq g(n) + h(n) = f(n)$$

$$f(n') \geq f(n)$$



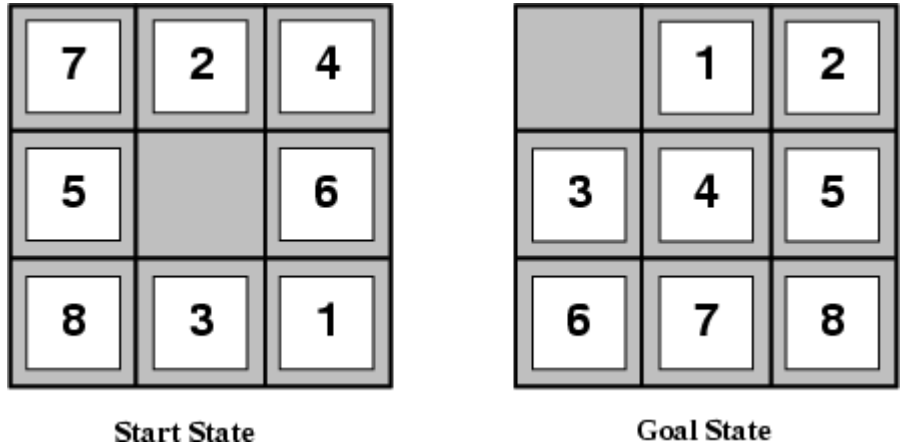
Theorem: If *h(n)* is consistent, A* using GRAPH-SEARCH is optimal

Properties of A*

- **Complete**? Yes (unless there are infinitely many nodes with f ≤ f(G))
- **Time**? Exponential
- **Space**? Keeps all nodes in memory
- **Optimal**? Yes

Effective heuristics

- E.g., for the 8-puzzle:



- h1(n) = number of misplaced tiles
- h2(n) = total Manhattan distance

(i.e., no. of squares from desired location of each tile)

- h1(S) = **8**
- h2(S) = **3+1+2+2+2+3+3+2 = 18**

Effective heuristics

- The total number of nodes generated by A∗ for a particular problem is **N** and the solution depth is **d** with the branching factor of **b***.

$$N = 1 + b^* + (b^*)^2 + \cdots + (b^*)^d$$

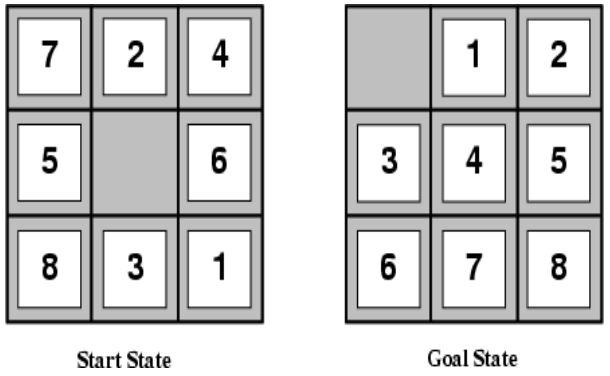
- b* should be close to 1 (**good heuristic**)
- if A∗ finds a solution at depth 5 using 52 nodes, then the effective branching factor is 1.92.

  Solve the following

$$1 + b^* + (b^*)^2 + (b^*)^3 + (b^*)^4 + (b^*)^5 = 52$$

  b*≈ 1.92

Effective heuristics



	Search Cost (nodes generated)			Effective Branching Factor		
d	IDS	A*(h_1)	A*(h_2)	IDS	A*(h_1)	A*(h_2)
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	–	539	113	–	1.44	1.23
16	–	1301	211	–	1.45	1.25
18	–	3056	363	–	1.46	1.26
20	–	7276	676	–	1.47	1.27
22	–	18094	1219	–	1.48	1.28
24	–	39135	1641	–	1.48	1.26

Figure 3.29 Comparison of the search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A* algorithms with h_1, h_2. Data are averaged over 100 instances of the 8-puzzle for each of various solution lengths d.

- E.g., for the 8-puzzle:
- h1(n) = number of misplaced tiles
- h2(n) = total Manhattan distance

Relaxed problems

- A problem with fewer restrictions on the actions is called a relaxed problem
- The cost of an optimal solution to a relaxed problem is an admissible heuristic for the original problem
- If the rules of the 8-puzzle are relaxed so that a tile can move anywhere, then $h_1(n)$ gives the shortest solution
- If the rules are relaxed so that a tile can move to any adjacent square, then $h_2(n)$ gives the shortest solution

References

- CH 3- Solving Problems by Searching (Section 3.5-3.6)